

Algorithm and Data Structure

Part 6: Dynamic Programming

Ali El Hadi Ismail Fawaz

ENSISA 1A Info, Université de Haute-Alsace



Une école d'ingénieurs de l'Université de Haute-Alsace



Table of Contents

- 1 Overview
- 2 Core Principles of Dynamic Programming
- 3 Longest Increasing Subsequence (LIS)
- 4 Shortest Path
- 5 Edit Distance (Levenshtein)

Overview

Dynamic Programming: Context & Impact

Where it came from:

- Introduced by **Richard Bellman**, an **American mathematician**, in the 1950s
- Developed at the RAND Corporation (USA) for operations research and optimization
- The name "*dynamic programming*" was chosen to make it sound practical and appealing to funding agencies

Why it matters:

- Converts exponential-time recursion into efficient polynomial-time solutions
- Core technique in algorithms, machine learning, bioinformatics, economics, and AI

Real-life example:

- *DNA/Protein sequence alignment* (edit distance / Smith–Waterman)
- Enables modern genomics: comparing genomes in hours instead of centuries
- Critical in personalized medicine, evolutionary biology, and pandemic response

“Those who cannot remember the past are condemned to repeat it.”

— George Santayana, *The Life of Reason* (1905)

Note for class

This quotation is often associated by analogy with Dynamic Programming, since DP “remembers” past computations to avoid repeating work. While Bellman himself may not have cited Santayana, the connection is widely used in teaching.

Core Principles of Dynamic Programming

When to Use Dynamic Programming?

Two key properties:

① **Overlapping Subproblems**

The same sub-computations appear repeatedly (ex: Fibonacci numbers)

② **Optimal Substructure**

An optimal solution can be built from optimal solutions to subproblems (ex: shortest paths)

Examples:

- Fibonacci sequence
- Shortest path in graphs (without negative cycles)
- Coin change problem

Two Ways to Apply Dynamic Programming

1. Memoization (Top-Down)

- Start with the original recursive solution
- Store (cache) results of subproblems the first time they are computed
- On repeated calls, return the stored result instead of recomputing
- Easy to write, often closer to the mathematical recurrence

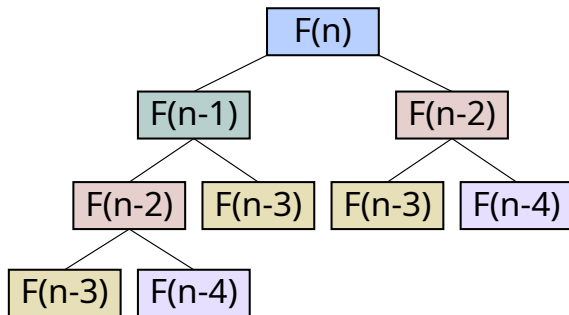
2. Tabulation (Bottom-Up)

- Build a table iteratively from base cases up to the final answer
- Ensures each subproblem is solved only once in the right order
- More space-efficient (no recursion stack)
- Often faster in practice due to better memory locality

Memoization = top-down with cache vs. Tabulation = bottom-up table

Solving Fibonacci with DP

$$F(n) = F(n-1) + F(n-2)$$



- Lots of repetitions

Solving Fibonacci with Top-Down DP

Top-Down DP Fibonacci(n)

Input: n , array DP of size $n+1$ filled with -1

Output: n^{th} element in the Fibonacci sequence

Instructions:

1. If $n \leq 1$
Return n
2. If $DP[n] \neq -1$
Return $DP[n]$
3. $DP[n] \leftarrow \text{Fibonacci}(n-1, DP) + \text{Fibonacci}(n-2, DP)$
4. Return $DP[n]$

- **Time Complexity** $\mathcal{O}(n)$ **Space Complexity** $\mathcal{O}(n)$
- Keep recursion
- Use memorization

Solving Fibonacci with Bottom-Up DP

Bottom-Up DP Fibonacci(n)

Input: n

Output: n^{th} element in the Fibonacci sequence

Instructions:

1. $DP \leftarrow [0] * n$
2. $DP[0] \leftarrow 0$
3. $DP[1] \leftarrow 1$
4. For i from 2 to $n + 1$ do
 $DP[i] \leftarrow DP[i - 1] + DP[i - 2]$
5. Return $DP[n]$

- **Time Complexity** $\mathcal{O}(n)$ **Space Complexity** $\mathcal{O}(n)$
- Remove recursion use iterative
- Use tabulation

Even More Optimization

Use the fact that $F(n)$ only depends on the two previous elements not all elements from 0 to $n - 1$

Bottom-Up DP Fibonacci(n)

Input: n

Output: n^{th} element in the Fibonacci sequence

Instructions:

1. $prev1, prev2, curr \leftarrow 0, 1, 1$
2. For i from 2 to $n + 1$ do
 - a. $curr \leftarrow prev1 + prev2$
 - b. $prev1 \leftarrow prev2$
 - c. $prev2 \leftarrow curr$
2. Return $curr$

- Time Complexity $\mathcal{O}(n)$ Space Complexity $\mathcal{O}(1)$

Even More Optimization

Use the fact that $F(n)$ only depends on the two previous elements not all elements from 0 to $n - 1$

Bottom-Up DP Fibonacci(n)

Input: n

Output: n^{th} element in the Fibonacci sequence

Instructions:

1. $prev1, prev2, curr \leftarrow 0, 1, 1$
2. For i from 2 to $n + 1$ do
 - a. $curr \leftarrow prev1 + prev2$
 - b. $prev1 \leftarrow prev2$
 - c. $prev2 \leftarrow curr$
2. Return $curr$

- **Time Complexity** $\mathcal{O}(n)$ **Space Complexity** $\mathcal{O}(1)$
- Can we be more optimal ? Yes, but more advanced, find it if you can :)

Longest Increasing Subsequence (LIS)

Longest Increasing Subsequence (LIS)

- **Problem statement:** Given a sequence of numbers, find the longest subsequence (not necessarily contiguous) that is strictly increasing
- **Example:**
 - Input: [3, 10, 2, 1, 20]
 - Output: 3
 - Explanation: LIS = [3, 10, 20]
- **Naive:** Try all subsequences recursively, keep the longest valid increasing one
- **Complexity of naive:** Exponential, $\mathcal{O}(2^n)$, since all subsequences are explored
- **Applications:**
 - Trend detection, signal processing, and pattern recognition
- **How DP helps:** Dynamic Programming reuses overlapping subproblems and computes LIS in $\mathcal{O}(n^2)$ (or even $\mathcal{O}(n \log n)$ with advanced methods).

Naive Solution for LIS

Naive_LIS_idx

Input: Array A of size n , int idx , Output: LIS ending at idx

Instructions:

1. If $idx == 0$
 Return 1
2. $res \leftarrow 1$
3. For $prev$ from 1 to $idx - 1$ do
 - a. If $A[prev] < A[idx]$
 $res \leftarrow \max(res, 1 + Naive_LIS_idx(A, prev))$
4. Return res

Naive_LIS

Input: Array A of size n , Output: LIS

Instructions:

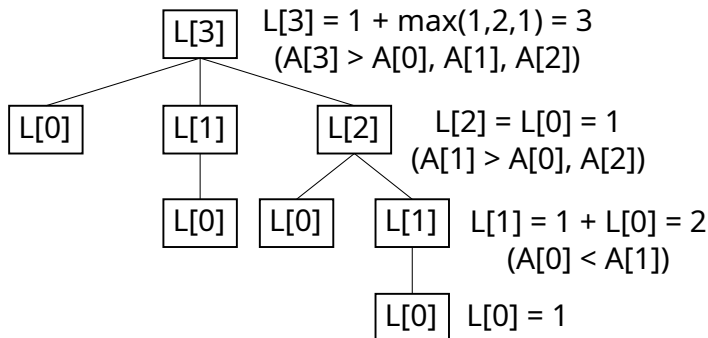
1. $res \leftarrow 1$
3. For i from 2 to n do
 - a. $res \leftarrow \max(res, 1 + Naive_LIS_idx(A, i))$
4. Return res



Why Use DP for LIS ?

$A = [3, 10, 2, 11]$

$L = [0, 0, 0, 0]$



Bottom-Up DP for LIS

Naive_LIS

Input: Array A of size n , Output: LIS

Instructions:

1. $L \leftarrow [1] * n$ # Array of size n filled with 1s
2. For i from 1 to $n - 1$ do
 3. For $prev$ from 0 to $i - 1$ do
 - a. If $A[i] > A[prev]$
 $L[i] \leftarrow \max(L[i], 1 + L[prev])$
4. $res \leftarrow 1$
5. For i from 0 to $n - 1$ do
 - a. $res \leftarrow \max(res, L[i])$
6. Return res

- **Time Complexity:** $\mathcal{O}(n^2)$ **Space Complexity:** $\mathcal{O}(n)$

Bottom-Up DP for LIS

Naive_LIS

Input: Array A of size n , Output: LIS

Instructions:

1. $L \leftarrow [1] * n$ # Array of size n filled with 1s
2. For i from 1 to $n - 1$ do
 3. For $prev$ from 0 to $i - 1$ do
 - a. If $A[i] > A[prev]$
 $L[i] \leftarrow \max(L[i], 1 + L[prev])$
4. $res \leftarrow 1$
5. For i from 0 to $n - 1$ do
 - a. $res \leftarrow \max(res, L[i])$
6. Return res

- **Time Complexity:** $\mathcal{O}(n^2)$ **Space Complexity:** $\mathcal{O}(n)$
- Can we do it in $\mathcal{O}(n \log n)$ time ? Yes using Binary Search

Shortest Path

Shortest Path Problem with Bellman-Ford

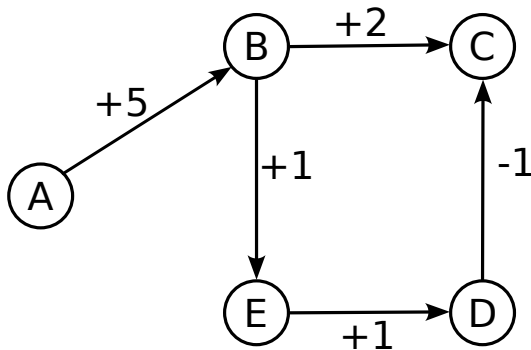
- **Problem statement:** Given a weighted graph $G = (V, E)$ and a source vertex s , find the shortest path from s to all other vertices (allowing negative weights, but no negative cycles)
- **Naive approach:**
 - Enumerate all possible paths from s to each vertex
 - Select the one with minimum total weight
 - **Complexity:** Exponential, since the number of paths can blow up

Shortest Path Problem with Bellman-Ford

- **Problem statement:** Given a weighted graph $G = (V, E)$ and a source vertex s , find the shortest path from s to all other vertices (allowing negative weights, but no negative cycles)
- **Naive approach:**
 - Enumerate all possible paths from s to each vertex
 - Select the one with minimum total weight
 - **Complexity:** Exponential, since the number of paths can blow up
- **Dynamic Programming approach (Bellman-Ford):**
 - Maintain an array $\text{dist}[v]$ of shortest distances from s
 - Initialize: $\text{dist}[s] = 0$, others $= \infty$
 - Repeat $|V| - 1$ times:
 - For each edge (u, v) with weight $w(u, v)$:

$$\text{dist}[v] = \min(\text{dist}[v], \text{dist}[u] + w(u, v))$$
 - After $|V| - 1$ iterations, all shortest paths are found
 - **Detecting negative cycles:** Perform one more relaxation, if any distance decreases, a negative cycle exists
 - **Time Complexity** $\mathcal{O}(V \cdot E)$ **Space Complexity** $\mathcal{O}(V)$
- **Applications:**
 - Network routing (finding optimal paths in communication networks)

Bellman-Ford Example

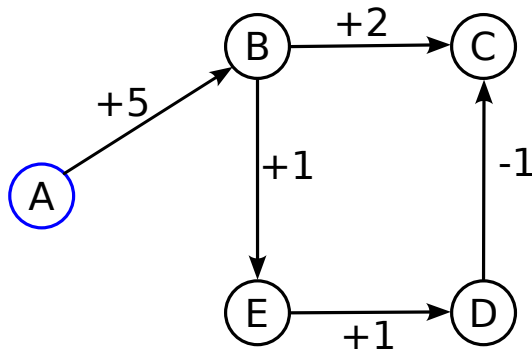


Source = A

dist:

$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$
A	B	C	D	E

Bellman-Ford Example

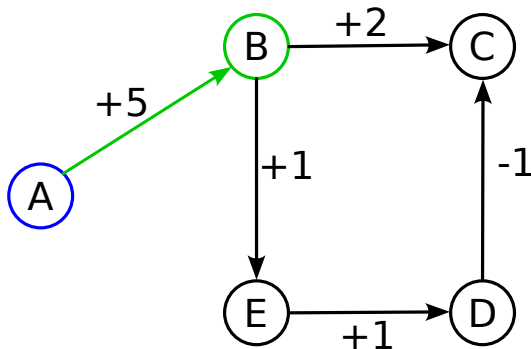


Source = A

dist:

0	$+\infty$	$+\infty$	$+\infty$	$+\infty$
A	B	C	D	E

Bellman-Ford Example

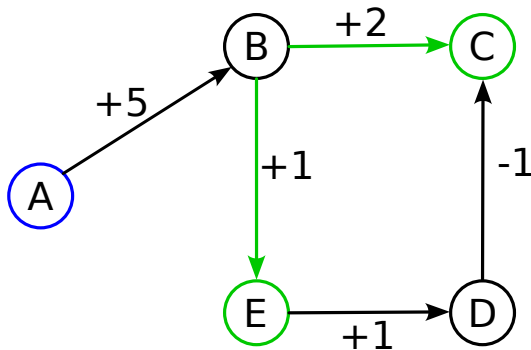


Source = A

dist:

0	+5	$+\infty$	$+\infty$	$+\infty$
A	B	C	D	E

Bellman-Ford Example

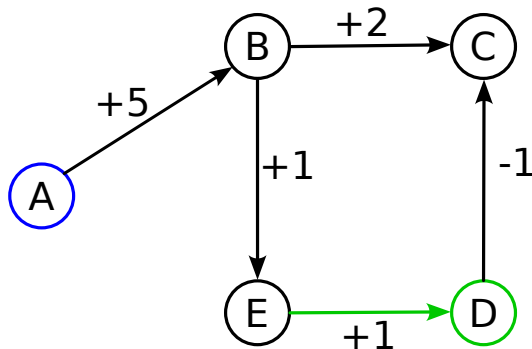


Source = A

dist:

0	+5	+7	+ ∞	+6
A	B	C	D	E

Bellman-Ford Example

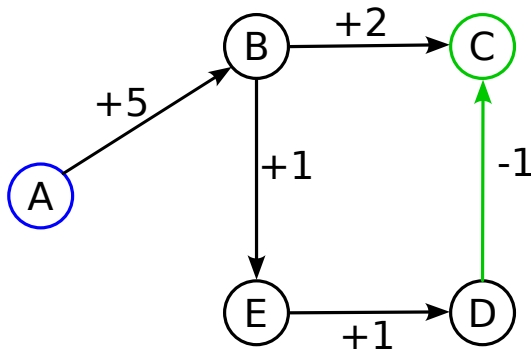


Source = A

dist:

0	+5	+7	+7	+6
A	B	C	D	E

Bellman-Ford Example

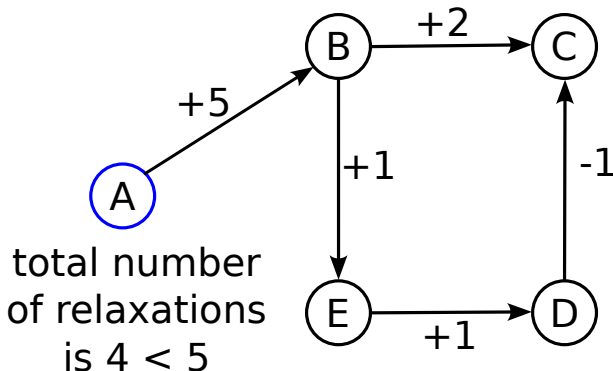


Source = A

dist:

0	+5	+6	+7	+6
A	B	C	D	E

Bellman-Ford Example



Source = A

dist:

0	+5	+6	+7	+6
A	B	C	D	E

Bellman–Ford Shortest Path Algorithm

Bellman_Ford

Input: $\text{int } V$ $\text{int } s$ $\text{list}[\text{list}[\text{int}]]$ $edges$, Output: Array $dist[v]$
shortest path to s

Instructions:

1. $dist \leftarrow [\infty] * V$ # Array of size V filled with ∞ s
2. $dist[s] \leftarrow 0$
3. For idx from 0 to $V - 1$ do
 - a. For edge in $edges$ do
 - i. $u, v, w \leftarrow edge[0], edge[1], edge[2]$
 - ii. If $dist[u] \neq \infty$ and $dist[v] > dist[u] + w$
 - If $idx == V - 1$
 - Return -1
 - $dist[v] \leftarrow dist[u] + w$
4. Return $dist$

Edit Distance (Levenshtein)

Edit Distance: Problem & Approaches

- **Problem statement:** Given strings s and t of size n and m respectively, compute the minimum number of edits to transform s into t , where edits are **insert**, **delete**, or **substitute** a single character
- **Example:** $s = \text{"abcd"}$, $t = \text{"bcfe"}$ $\Rightarrow$ 3 edits delete 'a', substitute 'd' with 'f' and insert 'e' at the end

Edit Distance: Problem & Approaches

- **Problem statement:** Given strings s and t of size n and m respectively, compute the minimum number of edits to transform s into t , where edits are **insert**, **delete**, or **substitute** a single character
- **Example:** $s = \text{"abcd"}$, $t = \text{"bcfe"}$ $\Rightarrow$ 3 edits delete 'a', substitute 'd' with 'f' and insert 'e' at the end
- **Naive approach:** Try all sequences of edits via recursion:
 - If last chars equal: solve on prefixes
 - Else: take $1 + \min(\text{delete}, \text{insert}, \text{substitute})$ on prefixes
 - **Complexity:** Exponential in time $\mathcal{O}(3^{n+m})$ (branches up to 3 per step), impractical for longer strings

Edit Distance: Problem & Approaches

- **Problem statement:** Given strings s and t of size n and m respectively, compute the minimum number of edits to transform s into t , where edits are **insert**, **delete**, or **substitute** a single character
- **Example:** $s = \text{"abcd"}$, $t = \text{"bcfe"}$ $\Rightarrow$ 3 edits delete 'a', substitute 'd' with 'f' and insert 'e' at the end
- **Naive approach:** Try all sequences of edits via recursion:
 - If last chars equal: solve on prefixes
 - Else: take $1 + \min(\text{delete}, \text{insert}, \text{substitute})$ on prefixes
 - **Complexity:** Exponential in time $\mathcal{O}(3^{n+m})$ (branches up to 3 per step), impractical for longer strings
- **DP approach (tabulation):**
 - Define array dp of size $(n+1) * (m+1)$ where $d[i][j]$ = edit distance between $s[: (i-1)]$ and $t[: (j-1)]$
 - **Base cases:** $dp[i][0] = i$, $dp[0][j] = j$
 - **Iterate:**

$$dp[i][j] = \begin{cases} dp[i-1][j-1], & \text{if } s[i-1] == t[j-1] \\ 1 + \min\{dp[i-1][j], dp[i][j-1], dp[i-1][j-1]\}, & \text{otherwise} \end{cases}$$

- **Complexity:** $O(n \cdot m)$ in time, $O(n \cdot m)$ in space

Edit Distance

Edit_Distance

Input: string s of size n , string t of size m

Output: int = minimum edits

Instructions:

1. $dp \leftarrow$ Array of $int[n+1][m+1] = \{0\}$
2. For $i \leftarrow 0$ to n do
 $dp[i][0] \leftarrow i$
3. For $j \leftarrow 0$ to m do
 $dp[0][j] \leftarrow j$
4. For $i \leftarrow 1$ to n do
 - a. For $j \leftarrow 1$ to m do
 - i. If $s[i-1] = t[j-1]$
 $dp[i][j] \leftarrow dp[i-1][j-1]$
 - ii. else
 $dp[i][j] \leftarrow 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$
5. Return $dp[n][m]$

Edit Distance Example

	b	c	f	e
a				
b				
c				
d				

Edit Distance Example

		b	c	f	e
	0	1	2	3	4
a	1				
b	2				
c	3				
d	4				

When one string is empty
the number of edits is the
length of the other string

Edit Distance Example

		b	c	f	e
	0	1	2	3	4
a	1				
b	2				
c	3				
d	4				

$$\min(0, 1, 1) = 0$$

Edit Distance Example

		b	c	f	e
	0	1	2	3	4
a	1	1			
b	2				
c	3				
d	4				

Edit Distance Example

		b	c	f	e
	0	1	2	3	4
a	1	1	2	3	4
b	2				
c	3				
d	4				

same for the rest of this line

Edit Distance Example

		b	c	f	e
	0	1	2	3	4
a	1	1	2	3	4
b	2	1			
c	3				
d	4				

Edit Distance Example

		b	c	f	e
	0	1	2	3	4
a	1	1	2	3	4
b	2	1	2	3	4
c	3				
d	4				

Edit Distance Example

		b	c	f	e
	0	1	2	3	4
a	1	1	2	3	4
b	2	1	2	3	4
c	3	2	1	2	3
d	4	3	2	2	3

we fill the rest following
the same methodology

Edit Distance Example

		b	c	f	e
	0	1	2	3	4
a	1	1	2	3	4
b	2	1	2	3	4
c	3	2	1	2	3
d	4	3	2	2	3

Edit distance = 3



Edit Distance Example

s = abcd

t = bcfe

		b	c	f	e
	0	1	2	3	4
a	1	1	2	3	4
b	2	1	2	3	4
c	3	2	1	2	3
d	4	3	2	2	3

backtrace

Edit Distance Example

s = abcd

t = bcf^e

		b	c	f	^e
	0	1	2	3	4
a	1	1	2	3	4
b	2	1	2	3	4
c	3	2	1	2	3
d	4	3	2	2	3

backtrace

- Delete e

Edit Distance Example

s = abcd

t = bcfe

		b	c	f	e
	0	1	2	3	4
a	1	1	2	3	4
b	2	1	2	3	4
c	3	2	1	2	3
d	4	3	2	2	3

backtrace

- Delete e
- Substitute f & d

Edit Distance Example

s = abcd

t = bcfe

		b	c	f	e
	0	1	2	3	4
a	1	1	2	3	4
b	2	1	2	3	4
c	3	2	1	2	3
d	4	3	2	2	3

backtrace

- Delete e
- Substitute f & d

Edit Distance Example

s = abcd

t = bcfe

		b	c	f	e
	0	1	2	3	4
a	1	1	2	3	4
b	2	1	2	3	4
c	3	2	1	2	3
d	4	3	2	2	3

backtrace

- Delete e
- Substitute f & d

Edit Distance Example

s = **a**bc**d**

t = bcf**e**

		b	c	f	e
a	0	1	2	3	4
b	1	1	2	3	4
c	2	1	2	3	4
d	3	2	1	2	3
	4	3	2	2	3

backtrace

- Delete e
- Substitute f & d
- Delete a

Edit Distance Example

		b	c	f	e
	0	1	2	3	4
a	1	1	2	3	4
b	2	1	2	3	4
c	3	2	1	2	3
d	4	3	2	2	3

optimal alignment path